

Authenticated Privacy-Preserving Deck Reconstruction for Mental Poker

Exact carrier coverage, slot semantics, and a machine-checked composition boundary

Preprint | September 25, 2026 | arXiv subject: cs.CR

Qining Lin

Qining Lin: linqining1994@gmail.com · ORCID 0009-0001-4347-8771

Source repository: github.com/linqining/poker_protocol

Article license: arXiv.org perpetual, non-exclusive license 1.0

Abstract

Mental Poker encrypts and shuffles a deck so that no single participant learns the complete order. A multiplayer shuffle normally requires every participant to act. If a participant leaves or stops responding, the remaining players must remove that participant's cards without learning or changing the other cards, but a shuffle argument alone does not close this liveness gap.

We present a residual-ciphertext reconstruction protocol based on authenticated, state-bound residual carriers obtained by subtracting submitted reveal tokens. With one missing token, the carrier is encrypted under the missing owner's key; with two or more, it is jointly keyed and not individually decryptable. Reconstruction can still proceed: only an authenticated owner residual authorizes a negative contribution, while a jointly keyed unknown card takes the zero branch and remains encrypted.

For each canonical slot, a prover contributes an aggregate-key encryption of zero or the negated slot card. A cross-key proof links negative branches to authenticated carriers; a hidden shuffle conceals the carrier-to-slot mapping; OR proofs enforce the slot relation; and a domain-separated transcript binds context, epoch, state, keys, cards, carriers, and contributions. Under the stated assumptions, an accepted concrete package yields an extractable witness with exact carrier coverage. Separately, composition is proved in a session-bound ideal NIZK hybrid; the current Fiat-Shamir implementation is excluded from that realization claim.

Keywords. Mental Poker; verifiable shuffle; ElGamal; authenticated reconstruction; non-interactive zero knowledge; formal verification.

1. Introduction and motivation

Mental Poker protocols use public-key encryption, re-encryption, and proofs of correct shuffling to let several players deal and play without revealing the deck. The basic multiplayer construction is cooperative: each player applies a private shuffle or masking step, and the next phase assumes that all required steps are completed. This assumption is reasonable for a synchronous toy protocol, but it is a serious availability problem for a real table. A player may disconnect, crash, or deliberately refuse to continue. If the cards still contain that player's encryption layer, the remaining players cannot simply delete the layer or guess which cards belonged to the absent player. The deck then cannot be safely reconstructed.

The central contribution of this work is to separate card provenance from the absent user's participation in the current round. During normal play, the table state records authenticated residual ciphertexts together with the reveal-token lineage that produced them. After one or more players fail to submit a token, an active participant can use each decryptable owner-residual carrier to construct a reconstruction proof. The proof removes exactly those authorized cards, preserves every other canonical card, and exposes neither the mapping nor the card values. A jointly keyed residual that no player can decrypt authorizes no negative branch, so its canonical card remains in the new deck. Thus the table can proceed

Contributions

TABLE I. Summary of contributions

No.	Contribution	Statement
1	Reveal-token derivation	We derive the residual carrier by subtracting submitted reveal tokens and distinguish the owner-residual and jointly keyed cases.
2	Liveness-aware reconstruction	Authenticated residual carriers allow an active table to rebuild after non-participation; a deadline is a no-op event, not a plaintext disclosure.
3	Aggregate-key encryption	All contributions use the same aggregate key P , so the reconstructed deck retains the standard ElGamal shape.
4	Cross-key negation proof	For an authenticated carrier and a negative contribution under P , the prover proves the required ciphertext relation without exposing the card or its mapping.
5	Per-slot OR semantics	Each canonical slot is proven to contain either an encryption of zero or an encryption of the slot's negative card.
6	State and transcript binding	The table context, epoch, previous-state digest, public keys, canonical cards, residual carriers, and contributions are bound into a domain-separated transcript.
7	Explicit hybrid model	We define F_RECON command by command and prove static-corruption composition in the F_NIZK^R_RECON hybrid without claiming a concrete Fiat-Shamir UC instantiation.
8	Machine-checked composition	Lean derives end-to-end package semantics from explicit component and refinement interfaces.

without requiring every previous card plaintext to be known by an online player.

Let the canonical deck be a public, duplicate-free sequence of curve points $M = (m_0, \dots, m_{n-1})$. A card is initially encrypted under the aggregate key $P = \sum_p Q_p$. For each residual derivation, let A be the set of players whose reveal tokens were submitted for that carrier and let U be the corresponding carrier-specific missing-token set; $|U|$ is not the total number of absent players. The residual is encrypted under $Q_U = \sum_{p \in U} Q_p$. The special case $|U| = 1$ is an owner-residual carrier; the general case $|U| \geq 2$ is jointly keyed. Reconstruction must preserve this distinction while ensuring that the owner-to-slot mapping and branch choices remain hidden and the output remains a valid ElGamal deck for the next shuffle.

The difficult part is slot semantics. A proof of a ciphertext multiset or of a global linear sum is insufficient: compensating ciphertexts can make the sum look correct while changing an individual slot. The protocol therefore proves the allowed plaintext relation independently for every slot, and separately proves that each negative branch is linked to an authenticated residual carrier.

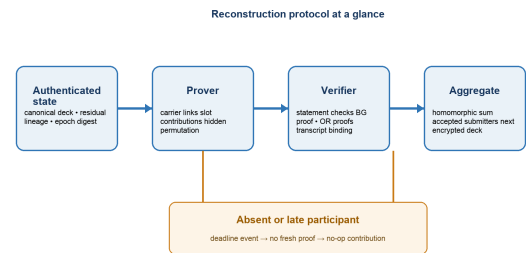


Figure 1. The reconstruction path separates authenticated provenance from current-round participation. A missed deadline contributes no fresh proof and therefore has no effect on the aggregate deck.

2. Background and related work

The original Mental Poker problem asked players to deal and play cards without revealing hidden state [41,42]. Practical constructions combine ElGamal encryption [15] with successive private shuffles; constant-round and communication-efficient protocols refine the full-game interface [45,46]. Bayer–Groth [2], Furukawa–Sako [22],

Neff [23], Groth [24], and universally composable mixnets [25] establish different efficiency and composition points for verifiable shuffling. Schnorr, Chaum–Pedersen, and partial-knowledge protocols provide the linear and disjunctive relations used here [3,4,6]. These components prove permutation or relation correctness; they do not authenticate which prior-state carrier authorizes removal from a new canonical deck.

Beaver’s 2025 study of extending Mental Poker addresses OT amplification/extraction, ordinary-deck base-OT construction, and two-player/card-cryptography foundations [49]. Its focus is therefore below the multiplayer reconstruction interface addressed here: it establishes a different primitive layer and does not state a state-bound residual-carrier language, per-slot zero-or-negative package relation, browser cost grid, or Lean composition boundary.

Fiat–Shamir [5], the random-oracle methodology [18], classical forking analyses [19], online extractors [20], and quantum-ROM analyses [21] give distinct security guarantees and should not be conflated with UC realization. CRS-based NIZK ranges from the original feasibility result [17] to pairing-based and simulation-sound systems [26,27]. Our UC statement therefore uses a session-bound ideal NIZK functionality for the aggregate relation. The Poseidon Fiat–Shamir code is evaluated as a concrete standalone implementation; this paper does not prove that it realizes that ideal functionality under concurrent scheduling.

Formal cryptographic developments such as EasyCrypt [28], CryptHOL [29], Jasmin [30], and HACLS* [31] illustrate progressively stronger links between proofs and executable code. Our Lean development is narrower but not purely paper-only: it checks the reconstruction algebra and composition boundary, and the Rust/Lean fixtures cross-check a complete statement schema and one seeded proof-instance schema. Group implementation, component proof-system guarantees, concrete Fiat–Shamir realization, full byte-level serialization refinement, and authenticated host state remain explicit obligations. This scope is materially different from a verified implementation claim.

Dropout-resistant secure aggregation [32,33] addresses a related liveness problem in collaborative encrypted state, although its output semantics are aggregate vectors rather than a hidden card permutation. TIFS systems such as Zilch [34], practical multi-party PSI [35], secure distributed modular exponentiation [36], and dynamic

group authentication [37] show the journal’s interest in implementations that connect proof systems, group protocols, and measured costs. WebAssembly deployment [38] and succinct circuit systems [39,40] motivate our browser and scoped Circom measurements, but they do not provide a functionally equivalent reconstruction baseline.

Barnett–Smart [9] supplies the widely reused ElGamal mental-poker foundation and shuffle verification. Kurosawa et al. [10] and Soo et al. [11] tolerate bounded absence or lazy updates using secret sharing and rearrangeable networks, but the fixed threshold and coalition recovery capability are security costs. The 21 protocol reduces complexity for composable card games without secret state [43], while Oozu, Ishii, and Tanaka study dropout tolerance with a small deposit and an optimal upper bound on the number of dropouts [44]. Newer financially enforced designs — Instantaneous Decentralized Poker [12], Kaleidoscope [13], and ROYALE [14] — strengthen collateral, penalties, and composable payment semantics. Their absence handling is primarily forfeiture, timeout, restart, or an economic policy; they do not derive per-carrier removal authorization from authenticated reveal-token lineage, nor do they prove the per-slot $\{0, -m_i\}$ relation central to reconstruction.

A separate line enforces the rules of trick-taking games. Bultel and Lafourcade define secure online Spades and prevent a player from violating hidden play constraints [47]. Bella et al. extend that model to games with cards set aside, use partially homomorphic ciphertexts and a generic shuffle argument, and report a Rust implementation [48]. That work addresses play legality and set-aside-card semantics; it does not reconstruct a next-round aggregate-key deck from authenticated residual carriers after a participant fails to submit reveal tokens.

The protocol layers are therefore complementary rather than interchangeable. A complete deployment may need a full dealing and rule-compliance protocol, an economic dropout policy, and the reconstruction layer defined here. Table II summarizes this boundary without treating incomparable runtime numbers as a ranking.

TABLE II. Position among secure card-game protocol layers

Protocol family	What it establishes	Boundary with this work
Foundations and efficient full games [9,41–43,45,46,49]	ElGamal deck representation, shuffling, dealing, OT/ card-cryptography foundations, round/ communication efficiency, and composable game execution.	Primarily assumes the required players or state transitions remain available; no authenticated residual-carrier removal semantics.
Dropout and financial enforcement [7,12–14,44]	Continues after absence or enforces deposits, penalties, payouts, and bounded dropout behavior.	Provides liveness or economic incentives, but not exact per-card singleton authorization or the zero-or-negative slot relation.
Trick-taking rule enforcement [47,48]	Hidden hand legality, play constraints, and set-aside-card game semantics.	Secures moves within a game rather than rebuilding the next aggregate-key deck from a prior authenticated state.
This reconstruction layer	Authenticated owner-residual lineage, exact coverage, hidden carrier-to-slot mapping, per-slot plaintext membership, and a Lean composition boundary.	Does not replace dealing, rule enforcement, payments, penalties, or a complete game lifecycle.

Those ingredients do not by themselves solve the departure problem. A shuffle proof can show that an output is a re-encrypted permutation of an input, but it does not say that a particular output slot contains either the original card or an identity correction. Nor does it identify which correction is authorized by the authenticated history of the table. Our protocol adds the missing reconstruction layer: hidden shuffling protects the mapping, the slot OR proof restricts plaintexts, the cross-key proof links a negative contribution to an authenticated owner-residual carrier, and the state digest authenticates the card’s provenance.

Earlier TTP-free routes either asked a departing player to reveal its secret layer or used secret sharing to tolerate a fixed number of absentees; the latter permits a sufficiently large coalition to recover the deck. The closest liveness construction [7] instead uses CDS partial-knowledge proofs and veto factors, so the remaining table can continue without the departed player’s cooperation.

To compare protocol boundaries without inventing runtime numbers, let N denote active players in [7], $d=52$ cards, and r prior dealing rounds. Dropout recovery regenerates the whole deck: each face-down card consists of N threshold-ElGamal components, for dN public components. Each player’s veto layer publishes d re-masking pairs and one non-veto CDS argument plus r veto

CDS arguments, each over d Chaum–Pedersen instances. The subsequent re-masking chain uses about dN^2 Chaum–Pedersen proofs, followed by a full Barnett–Smart shuffle argument over dN ciphertext components. The extended thesis [8] states that dropout-path efficiency still needs improvement and reports no dropout-specific proof-byte or runtime measurements.

The comparison below summarizes the boundary with the closest dropout-tolerant construction. The comparison concerns the security object being proved: the earlier work provides a valuable liveness mechanism, while this paper adds authenticated per-card authorization and a machine-checked semantic bridge.

TABLE III. Comparison with the closest dropout-tolerant construction

Property	Dropout-tolerant TTP-free Mental Poker [7]	This work
Dropout handling	Continues after a player leaves	Continues after deadline or crash
Post-departure deck action	Remove leaver key share and regenerate; leaver’s cards return	Submit a state-bound package; singleton residuals removed, jointly keyed residuals retained
Public deck size	N ciphertext components per card, dN total	d canonical contributions plus k authenticated carriers
Proof relation count	$N(r+1)$ CDS arguments over d CP instances; about dN^2 chain CP proofs; then a full shuffle proof	k cross-key proofs + one Bayer–Groth + d slot OR proofs
Removal authorization	Protocol-level dropout recovery	Authenticated singleton residual-carrier lineage
Per-slot plaintext relation	Not stated as zero-or-negative membership	OR proof for 0 or $-m_i$ at every slot
Mapping privacy	Hidden by protocol components	Hidden carrier-to-slot map plus BG proof
Formal assurance	Cryptographic proof in the paper	Lean checked composition boundary plus explicit assumptions
Multi-missing-key case	Not separated as a carrier class	Jointly keyed residual retained; no unauthorized removal
Measured evidence	Symbolic description only; no dropout runtime or proof bytes	Native and WASM d,k grids; $d=52,k=13$ bundle is 27.75 KB

3. Model and assumptions

Let q be a large prime, F_q the scalar field, and G a prime-order additive group with generator g . For a public key P

= xg , define the additive ElGamal encryption and decryption functions as follows.

$$\text{Enc}_p(m; r) = (rg, m + rP), \quad \text{Dec}_x(C) = C.c_2 - xC.c_1.$$

ElGamal is additively homomorphic:

$$\text{Enc}_p(m; r) + \text{Enc}_p(m'; r') = \text{Enc}_p(m + m'; r + r').$$

The aggregate public key is $P = \sum_p pk_p$, and $Q_p = sk_p g$. Canonical cards are nonzero and pairwise distinct. The protocol relies on the following assumptions; the distinction between algebraic facts and computational assumptions is part of the security claim.

3.1 Reveal-token algebra and the origin of the reconstruction carrier

We use residual carrier for the general object, and owner-residual carrier only for the single-missing-token specialization. The security invariant is a residual ciphertext derived from an authenticated reveal-token transcript. Let the fully masked card be

$$C_i = \text{Enc}_p(m_i; r_i) = (r_i g, m_i + r_i P), \quad P = \sum_p Q_p.$$

For player p , the reveal token is the first ciphertext component multiplied by that player's secret key:

$$t_{p,i} = sk_p C_i.c_1 = r_i Q_p.$$

The Chaum–Pedersen/DLEQ proof attached to $t_{p,i}$ establishes that the same secret key links $Q_p = sk_p g$ and $t_{p,i} = sk_p C_i.c_1$. Therefore a verifier may subtract only tokens that are authenticated for the current card and epoch. Let A be the set of players whose reveal tokens were submitted for this carrier, and let $U = \text{Players} \text{ minus } A$ be its carrier-specific missing-token set. Define $Q_U = \sum_{p \in U} Q_p$. The residual carrier is

$$C_i \wedge (A) = C_i - \sum_{p \in A} t_{p,i} = (r_i g, m_i + r_i \sum_{p \in U} Q_p) = \text{Enc}_{\{Q_U\}}(m_i; r_i), \quad Q_U = \sum_{p \in U} Q_p.$$

The derivation is a direct group identity, but its interpretation depends on the carrier-specific value of $|U|$. If $|U| = 1$, say $U = \{q\}$, then $Q_U = Q_q$ and the residual carrier is decryptable by q ; this is the owner-residual carrier specialization used to authorize removal. If $|U| \geq 2$, then Q_U is a sum of several public keys. The remaining online players do not possess the aggregate secret key $sk_U = \sum_{p \in U} sk_p$, so no individual player learns m_i from the residual. This does not make reconstruction inapplicable. The protocol does not need to decrypt or remove that card: reconstruction starts from a fresh canonical base $B_i = \text{Enc}_P(m_i; i+1)$, and because no

owner-residual proof authorizes the negative branch for m_i , every accepted contribution for that slot encrypts zero. Homomorphic aggregation therefore retains m_i in the new deck. The old jointly keyed ciphertext can be discarded after its authenticated state transition has determined that no removal is authorized. If U is empty, all tokens were submitted and the protocol crosses the plaintext/redeal boundary rather than producing a residual carrier.

This retention rule is a deck-level safety policy, not a claim that every card game accepts the retained card in its next round. It preserves a well-formed ElGamal deck and prevents an unauthorized party from learning or deleting the jointly unknown card. An application must still define whether that canonical card may be dealt again, set aside, or removed; if removal is mandatory while no individual player knows the plaintext, the deployment needs a threshold authorization policy in addition to the present protocol.

The same derivation also explains why authenticated lineage is the foundation of reconstruction. An owner-residual carrier is not an arbitrary ciphertext supplied by a prover: its first component, masking randomness, owner key, card slot, epoch, and submitted-token set are fixed by the prior state. Consequently, the proof can safely establish the relation

$$Q_U = sk_U g, \quad S_j.c_1 = v_j g, \quad sk_U R_j.c_1 + v_j P = R_j.c_2 + S_j.c_2, \quad S_j = \text{Enc}_p(-m_{i(j)}; v_j).$$

For $|U| = 1$, sk_U is the owner secret key and the current code checks the relation after owner decryption. For $|U| \geq 2$, the current protocol deliberately does not instantiate this negative relation: the slot takes the zero branch and the canonical card is retained. Thus reconstruction remains live even when every online player is unable to read the old ciphertext. A deployment that instead wished to remove a jointly unknown card would need a distributed generalized-Schnorr or threshold relation proof; that stronger removal policy is a future extension, not a completeness requirement of the present construction.

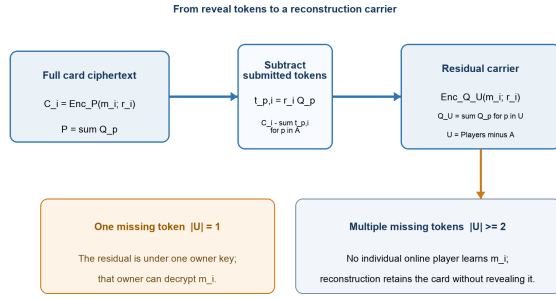


Figure 2. Reveal-token subtraction produces a residual ciphertext. One missing token gives an owner-residual specialization; multiple missing tokens give a jointly keyed carrier that remains usable for reconstruction without becoming individually decryptable.

TABLE IV. Security assumptions and operational meaning

ID	Assumption	Operational meaning
A1	Group and encoding	The curve implementation accepts only the prime-order subgroup and canonical encodings, and implements group operations correctly.
A2	Encryption privacy	ElGamal is IND-CPA secure under DDH or an equivalent assumption.
A3	Residual privacy	Every jointly keyed residual retains at least one honest, secret, uniform masking layer unless the protocol intentionally enters the plaintext/redeal path; privacy uses the corresponding fresh-discrete-log hardness assumption.
A4	Concrete hidden shuffle	For the standalone Rust theorem only, the Bayer–Groth component is complete, knowledge sound, and zero knowledge.
A5	Concrete linear proofs	For the standalone Rust theorem only, the cross-key and slot OR components are complete, specially sound, and HVZK; extracting a whole package requires their component extractors at their own Fiat–Shamir checkpoints.
A6	Ideal NIZK hybrid	Ordinary calls bind sid and the complete statement and accept only a valid aggregate witness. A simulator-only honest-proof interface and corrupted-proof witness interface are available only in the ideal comparison. No concrete Fiat–Shamir-to-UC realization is claimed.
A7	Authenticated state and refinement	The previous state digest cannot be forged, reveal-token proofs are bound to the card and epoch, and the Rust/AIR byte encoding refines the Lean statement.
A8	Authenticated missing-key sets	A negative contribution is accepted only for an authenticated singleton missing-token set. A residual with two or more missing keys authorizes only the zero branch, so its card remains in the rebuilt deck.
A9	Corruption model	Corruption is static, and an honest owner key or residual masking layer is not revealed before the execution ends.

AUTHENTICATED PRIVACY-PRESERVING DECK RECONSTRUCTION

Table V separates fixed implementation parameters from symbolic advantage terms. It does not assign numeric values to ϵ_{DDH} , ϵ_{state} , ϵ_{ser} , or ϵ_{KS} ; those remain reductions to the corresponding assumptions. The table also does not claim a conventional λ -bit security level for the hash-to-scalar maps, because the production challenge is a field-output reduction rather than rejection sampling.

TABLE V. Concrete parameters and theorem error boundaries

Parameter or bound	Value and scope
Production scalar order q	0x0800000000000010ffffffffffffb781126dcae7b2321e66a241adc64d2f, a 252-bit order; canonical scalars use 32-byte wire encoding.
BN254 G1 scalar order q	0x30644e72e131a029b85045b68181585d2833e84879b9709143e1f593f0000001, a 254-bit order in the scoped curve-level baseline.
Production challenge derivation	Poseidon transcript output is reduced modulo q . The domain labels and challenge ratchet are pinned by test vectors; the ideal-NIZK theorems do not claim that this map realizes concurrent UC NIZK.
BN254 challenge derivation	SHA3-256 output with the top three bits cleared, yielding a value below $2^{253} < q$; this baseline domain is separate from the production Poseidon path.
Completeness error	At most $O((n+k)q_H/q)$ at the stated zero-challenge boundary, with q the production order and n, k the deck/carrier dimensions.
Ideal UC distinguishing bound	At most $k \cdot \epsilon_{\text{DDH}} + \epsilon_{\text{state}} + \epsilon_{\text{ser}}$ for static corruption under A2 and A3, A6–A9. No FS or random-oracle term enters this hybrid theorem.
Non-owner veto bound	At most $\epsilon_{\text{state}} + \epsilon_{\text{ser}}$ in the ideal hybrid; the concrete verifier adds package-extraction error ϵ_{KS} under A1, A4, and A5.
Settlement transaction signature	Stark Schnorr uses a 32-byte public key and 64-byte signature over the domain-separated simulator message; it is test-model authenticity, not a deployed chain adapter.
Measured wire shape at $n=52, k=13$	Production/BN254 Borsh shape: 21,781-byte proof, 5,969-byte statement, and 27,750-byte statement-plus-proof bundle.
Frozen refinement schema	1,041-byte statement schema and 3,829-byte seeded proof-instance schema; fresh proofs remain randomized and are not canonicalized.

4. Protocol

4.1 Public statement

The public statement is

$$S = (\text{context_digest}, \text{epoch}, D_{\text{prev}}, P, Q, (m_i)_{i(n)}, (R_j)_j, (C_i)_{i(n)}).$$

Here R_j are authenticated residual ciphertexts and C_i are the prover’s contributions. The verifier rejects malformed lengths, identity keys or ciphertexts, duplicate canonical

cards, an empty carrier set, and a carrier set larger than the deck. The semantic relation is determined by these fields and by the reveal-token lineage that derives them; wire-level validation is fail-closed.

4.2 Proof generation

The prover has the authenticated owner-residual vector $R_0, \dots, R_{k-1}$ consisting precisely of carriers whose carrier-specific missing-token set has $|U| = 1$ and whose singleton owner is controlled by that prover. It also has the state-bound token derivation and the owner secret key. This vector is intentionally not the complete set of authenticated residual carriers. A jointly keyed carrier with $|U| \geq 2$ is omitted from this removal-authorizing vector; its canonical slot remains in the deterministic zero-padding path, so reconstruction still applies and preserves the encrypted card. Deleting such a jointly unknown card would require a separate threshold proof and authorization policy. The prover performs the following steps.

1. Derive each R_j by subtracting the authenticated submitted tokens; in the owner-residual specialization, decrypt it and check that its plaintext is a distinct canonical card.
2. Sample v_j and construct $S_j = \text{Enc}_p(-m_{i(j)}; v_j)$; construct $n-k$ deterministic zero contributions.
3. Apply a hidden permutation and fresh rerandomization to obtain the canonical contribution vector.
4. Prove the carrier-to-contribution relation for each pair (R_j, S_j) ; the one-owner path uses the cross-key Chaum–Pedersen relation described below.
5. Prove a Bayer–Groth relation from the hidden contribution vector to C .
6. Prove the two-branch OR relation for every canonical slot.

The carrier-to-slot mapping, branch choices, permutation, and randomness are prover witnesses and do not appear in the proof bytes.

4.3 Cross-key joint proof

For the owner-residual specialization, $R = \text{Enc}_Q(m; r)$ and $S = \text{Enc}_P(-m; v)$, and the prover shows knowledge of (sk_Q, v) satisfying

$$Q = \text{sk}_Q g, \quad S.c_1 = vg, \quad \text{sk}_Q R.c_1 + vP = R.c_2 + S.c_2.$$

AUTHENTICATED PRIVACY-PRESERVING DECK RECONSTRUCTION

This is a two-scalar generalized-Schnorr relation over three linked group equations. The third equation implies that the two ciphertext plaintexts sum to zero, so the negative contribution is tied to the authenticated owner-residual plaintext. The witness does not contain $r = \text{DL}(R.c_1)$. The relation is instantiated only when the carrier-specific set U is a singleton. For $|U| \geq 2$, no individual player can supply sk_U , so the protocol authorizes no negative contribution for that carrier; retaining the zero branch is sufficient to rebuild the deck without learning the card.

4.4 Slot OR proof

For slot i , define $T_0 = C_i.c_2$ and $T_1 = C_i.c_2 + m_i$. The prover shows that there is a v_i such that

$$C_i.c_1 = v_i g \quad \text{and} \quad T_b = v_i P \quad \text{for one branch } b \in \{0, 1\}.$$

Branch 0 is the zero contribution and branch 1 is the negative-card contribution. A standard OR proof honestly proves one branch, simulates the other with a challenge share, and enforces $e_0 + e_1 = e$ for the global challenge.

4.5 Aggregate reconstruction

Starting from the canonical base deck $B_i = \text{Enc}_p(m_i; i+1)$, the host sums every contribution whose proof passes before the deadline:

$$\tilde{B}_i = B_i + \sum_{p \in S_{\text{submit}}} C_{p,i}.$$

An absent or late participant contributes nothing. Under A8, at most one accepted contribution can carry the negative branch for a given authenticated residual slot. A participant cannot use a missed submission to authorize a negative contribution for a carrier outside the authenticated missing-key set.

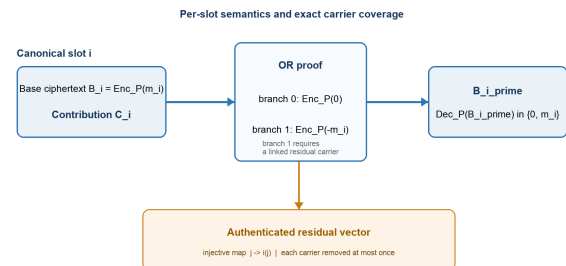


Figure 3. Slot-local OR semantics and exact residual-carrier coverage. The global deck relation is obtained by composing these per-slot constraints with an injective carrier-to-slot map.

4.6 Proof-system design rationale

The protocol deliberately retains Bayer–Groth for permutation hiding. It is the mature shuffle argument used by the implementation, and replacing it with a generic circuit compiler would change both the trusted-setup boundary and the implementation stack rather than isolate the new reconstruction semantics. The client-side construction therefore composes Bayer–Groth with cross-key Sigma proofs and slot OR proofs.

The contribution is this semantic composition, not a claim that the individual proof engines are new: authenticated residual-carrier lineage, a cross-key negative relation, exact coverage, and a zero-or-negative relation for every canonical slot. Transparent Sigma algebra also maps directly to the Lean component interface and keeps browser proving within the measured sub-second envelope. The resulting proofs are larger than a succinct aggregated argument; host-side aggregation remains future engineering work and is not used as an unmeasured performance comparison.

4.7 Deadline and deposit policy boundary

A pure application policy can make absence costly without changing the cryptographic reconstruction relation. The implementation models one required package per listed submitter in one epoch. Each listed submitter locks a deposit D ; after the cryptographic verifier accepts that submitter’s package, the host records its identity, proof digest, epoch, and submission time. Unknown submitters, duplicate submissions, wrong epochs, all-zero proof digests, and submissions after the deadline are rejected. The policy itself does not verify proofs and does not transfer funds; it emits a deterministic settlement proposal for an external settlement layer.

Let M be the set of listed submitters with no accepted package at the deadline. For each p in M , the proposed penalty is $a_p = \min(D, P)$, where P is the configured permissed-package penalty. Let $L = \sum_{p \in M} a_p$ and let T be the canonical-order list of timely submitters. If T is nonempty, each timely submitter receives $\text{floor}(L/|T|)$, the first $L \bmod |T|$ timely submitters receive one additional unit, and any remaining units are burned. If T is empty, L is burned. A missed submitter receives $D - a_p$; a timely submitter therefore releases its refund plus compensation. The no-claim and over-issue failure modes are excluded by construction: total compensation plus burned remainder equals L , and penalties never exceed locked

deposits. If all packages arrive before the deadline, every deposit is refunded and no penalty is proposed.

The settlement adapter boundary turns this proposal into auditable ledger events without requiring a particular chain API. Before finalization, the adapter locks each listed submitter’s exact deposit; an identical retry returns the same lock receipt, while a different amount is rejected. When applying a settlement, it requires submitter names to be unique and canonically ordered, checks $\text{refund} + \text{penalty} = \text{locked deposit}$ and $\text{refund} + \text{compensation} = \text{net release}$ for every player, verifies that the missed-submitter list is exact, and enforces both compensation and locked-escrow conservation equations. The escrow set must match the settlement exactly: no missing participant, extra participant, or amount mismatch is accepted. Application is atomic; an identical replay returns the original execution, while a different proposal for the same epoch is rejected.

A separate simulated chain host makes the transaction boundary explicit without pretending to be a blockchain. A settlement request carries a nonzero deployment digest and a Stark-curve Schnorr signature over that digest, compressed public key, nonce, gas limit, and complete proposal; the simulator verifies the signature and derives the account from the verified key. The sender field must be the canonical lowercase encoding of that key; identity public keys and identity Schnorr commitments are rejected. A versioned PSTX reference envelope canonicalizes the complete signed request for an off-chain adapter; it is not a public chain standard or deployed contract format. Submission requires the account’s next nonce and a gas limit at least equal to deterministic base-plus-event gas. The candidate execution is isolated from the committed ledger until a configured confirmation depth; an identical pending transaction is idempotent, while a different transaction or reused nonce is rejected. A reorg before finality drops the candidate, restores the nonce, and leaves the original escrow unchanged. The simulator therefore covers authenticity, canonical sender/key binding, signature tamper rejection, cross-deployment replay rejection, and rollback semantics, but mempool propagation, production gas accounting, gas refunds, host-consensus finality, and a deployed contract adapter remain concrete-host responsibilities.

This policy supplies economic liveness incentives, not a replacement owner witness. A replacement submitter cannot produce an owner-residual proof for another player

under the current v3 protocol. Supporting replacement or threshold authorization for a jointly keyed carrier requires a separate threshold relation and authentication policy; it is intentionally not smuggled in through deposit accounting.

5. Correctness and standalone security

Theorem 1 (Completeness)

If the statement satisfies the authenticated-state conditions and an honest prover follows Section 4.2, the verifier accepts except for explicit zero-challenge events, with failure probability bounded by $O((n+k)q_H/q)$.

Proof. A witness exists for every component. Lineage gives $R_j = \text{Enc}_Q(m_{i(j)}; r_j)$. The honest prover decrypts the unique canonical card, samples v_j , and constructs $S_j = \text{Enc}_P(-m_{i(j)}; v_j)$. Substituting in the third cross-key equation gives $\text{sk}_Q(r_j g) + v_j P = r_j Q + v_j P$ and $m_{i(j)} + r_j Q - m_{i(j)} + v_j P = r_j Q + v_j P$, so all three equations hold. The deterministic zero encryptions are $Z_1 = \text{Enc}_P(0; 1+1)$, and the selected injective map defines a permutation and rerandomizers; hence a complete Bayer–Groth witness exists.

For slot i , a zero input satisfies $C_i.c_1 = v_i g$ and $T_0 = C_i.c_2 = v_i P$, while a negative input satisfies $T_1 = C_i.c_2 + m_i = v_i P$. The real OR branch is answered honestly; the simulated branch computes its commitment from a chosen challenge share and response, and the two shares sum to the global challenge. Every statement field enters the transcript in the same canonical order. Failure therefore requires a zero challenge/share or a previously queried programmed point; across k cross-key proofs and n OR proofs the union is $O((n+k)q_H/q)$.

Theorem 2 (Knowledge soundness)

In a real $F_{\text{NIZK}^R\text{RECON}}$ -hybrid execution under A6 and A7, where the simulator-only interface is not invoked, ordinary acceptance yields a witness (removed , v , carrierIndex , ...) such that: (i) every slot contribution encrypts either 0 or $-m_i$; (ii) $\text{removed}_i = \text{true}$ exactly when some authenticated carrier index maps to slot i ; (iii) carrierIndex is injective; and (iv) every negative branch is linked to the authenticated residual-token derivation. For the concrete Rust verifier, the same conclusion is conditional on A1, A4, A5, and A7 and on a package

AUTHENTICATED PRIVACY-PRESERVING DECK RECONSTRUCTION extractor that invokes each component extractor at its own Fiat–Shamir checkpoint.

Proof. In the hybrid theorem, $F_{\text{NIZK}^R\text{RECON}}$ accepts only if a supplied witness satisfies the aggregate relation, so the four properties follow directly from its exact-coverage, shuffle, cross-key, and slot-membership clauses. In the concrete corollary, the Bayer–Groth extractor returns a permutation and rerandomizers; each cross-key extractor returns (sk_Q, v_j) ; and each slot extractor returns a branch and randomness. Because the implementation uses a cumulative transcript with distinct sequential challenges, this corollary does not claim that changing one final challenge extracts all components.

Combining the extracted shuffle with the OR witnesses yields $\text{removed}_i = \text{true}$ exactly when an extracted carrier maps to i . The prover rejects duplicate residual plaintexts and the extracted shuffle permutation is injective, so carrierIndex is injective. A7 and the byte refinement bind each accepted R_j to the authenticated prior hand; otherwise the reduction forges the state digest or violates serialization. Extractor failure is counted in ϵ_{KS} , and the state/encoding failure is counted in $\epsilon_{\text{state}} + \epsilon_{\text{ser}}$.

Theorem 3 (Reconstruction semantics)

Let $\chi_i = 1$ exactly when an accepted submitter has an authenticated owner-residual carrier that authorizes removal of m_i in the current epoch. Under A8,

$$\text{Dec}_p(\tilde{B}_i) = 0 \text{ if } \chi_i = 1; \quad \text{Dec}_p(\tilde{B}_i) = m_i \text{ if } \chi_i = 0.$$

Proof. Apply Theorem 2 to every accepted submitter. Each contribution for slot i encrypts 0 or $-m_i$, and only an authenticated carrier mapped to i can produce the negative plaintext. By A8, cross-player carrier sets are disjoint, so at most one accepted contribution is negative. ElGamal homomorphism gives $\text{Dec}_P(\tilde{B}_i) = m_i + \sum \text{plaintext}(C_{p,i})$, which is 0 when an authorized carrier exists and m_i otherwise. For a carrier-specific $|U| \geq 2$, no owner-residual witness is placed in the removal-authorizing vector, so every accepted contribution for its slot is zero and the card remains encrypted without being decrypted.

6. Ideal functionality and composition boundary

6.1 Hybrid model

The composition result is stated in the $F_{\text{NIZK}^R\text{RECON}}$, F_{STATE} , F_{KEY} , F_{SHUFFLE} ,

and F_REVEAL hybrid. On its ordinary interface, $F_NIZK^R_RECON$ binds sid and the complete public statement and accepts only if a witness satisfies the aggregate relation in Sections 4 and 5. In the ideal comparison only, an honest-proof simulator interface may register a session-bound simulated proof without revealing a witness, while a corrupted-proof interface supplies the accepted witness to the simulator. These capabilities are unavailable to the environment and ordinary protocol parties. This removes random-oracle programming and rewinding from the UC proof. The concrete Poseidon Fiat–Shamir implementation is outside this realization claim. The corruption set is fixed at initialization; the network adversary may reorder, delay, or drop submissions before the deadline.

6.2 Ideal functionality F_RECON

The session identifier is $sid = (\text{context}, \text{table}, \text{hand}, \text{epoch}, D_{\text{prev}})$. F_RECON stores $\text{phase} \in \{\text{WAITING}, \text{FINAL}\}$, the authenticated carrier authorization A_p for each player, a set of accepted submitters, and the final encrypted deck. F_STATE supplies the canonical deck, public keys, reveal-token lineage, missing-key sets, and epoch. Honest plaintexts, carrier-to-slot maps, proof witnesses, permutations, and rerandomizers are not leaked.

Input	From	Effect
INIT(sid , deadline)	F_STATE	Reject if sid is already active, D_{prev} is invalid, or the epoch is not the unique successor. Fix the static corruption set and authenticated A_p values; publish only public metadata; set $\text{phase}=\text{WAITING}$.
SUBMIT(sid, p, x)	Party or adversary	Reject wrong sid/epoch , FINAL phase, duplicates, malformed x , or failed $F_NIZK^R_RECON$ validation. Otherwise record p and its public contribution vector. Arrival order has no semantic effect.
STATUS(sid)	Environment	Return WAITING/FINAL, accepted identities, public validation results, deadline state, and public digests. Do not return witnesses, hidden maps, branches, or plaintexts.
DEADLINE(sid)	Clock or adversary	Validate that accepted negative branches equal the submitting players' authenticated singleton authorizations. On overlap or inconsistent state output STATE_INVALID. Otherwise aggregate accepted contributions, retain multi-key residual cards, publish the new encrypted deck and digest, and set $\text{phase}=\text{FINAL}$.
REPLAY or duplicate	Any caller	A command with a stale epoch, different D_{prev} , finalized sid , or repeated submission identifier is rejected without changing state. Repeated STATUS and DEADLINE after FINAL return the recorded public result.

For a statically corrupted player, the adversary chooses all local inputs and the simulator observes the witness submitted to $F_NIZK^R_RECON$ while emulating that corrupted interface. Honest witnesses remain hidden. This timing is the only point at which the simulator learns a corrupted removal map. Deadlines reveal no additional residual information. Cross-epoch and cross-table replay fails because sid includes context, table, hand, epoch, and D_{prev} .

6.3 Hybrid protocol

The client obtains the exact owner-residual vector and state binding, forms the public contribution vector, and invokes $F_NIZK^R_RECON$ on the aggregate relation. The host accepts only a successful functionality result with the current sid , then performs homomorphic aggregation. A carrier with $|U| \geq 2$ never appears in the removal-authorizing vector; its zero branch is nevertheless a normal reconstruction path. An abort or missed deadline is a no-op, not an arbitrary negative contribution.

6.4 Composition theorem

Theorem 4. Under A2, A3, and A6–A9, the hybrid protocol UC-realizes F_RECON against static corruption. When the authenticated-state and byte-refinement layers

are included as computational implementations, the distinguishing advantage is at most $k \cdot \epsilon_{\text{DDH}} + \epsilon_{\text{state}} + \epsilon_{\text{ser}}$. There is no Fiat–Shamir forking, random-oracle programming, or concurrent-FS error term in this theorem.

Simulator and hybrids

The simulator runs the adversary internally and relays public metadata and scheduling events. For an honest submission it publishes the ideal state adapter’s simulated contribution vector and invokes the simulator-only honest-proof interface; the ideal state retains the prescribed removal semantics. For a corrupted submission it reads the witness supplied by the corrupted-proof interface to $F_{\text{NIZK}^R_{\text{RECON}}}$, verifies the carrier map against F_{STATE} , and sends exactly the authenticated singleton removal set to F_{RECON} . Invalid, duplicate, late, replayed, and overlapping submissions receive the same public result in both executions.

H_0 is the hybrid protocol. In H_1 , replace each honest $\text{Enc}_P(-m_i; v_i)$ contribution by $\text{Enc}_P(0; v'_i)$ and register its proof through the simulator-only interface. A hybrid over at most k authorized negative contributions uses one IND-CPA reduction per replacement, for total cost $k \cdot \epsilon_{\text{DDH}}$; simulated public ciphertexts remain linked to the ideal semantic state only through F_{STATE} and are never used as extracted corrupted inputs. In H_2 , replace the state and serialization adapters by their ideal authenticated interfaces, at costs ϵ_{state} and ϵ_{ser} . H_2 is the ideal F_{RECON} execution because accepted corrupted witnesses specify exactly the authorized removal set and all honest hidden values are maintained by the functionality. Summing the hops gives the stated bound. The UC composition theorem applies to the named ideal functionalities; it does not establish a concrete realization of F_{NIZK} by the current Rust proof bytes.

6.5 Vetoing another player’s card

Define $\text{Veto}(p, m)$ as an accepted package by player p that removes m even though the authenticated residual-carrier derivation for p ’s epoch does not authorize m .

Theorem 5. Under A6–A9, $\Pr[\text{Veto}(p, m)] \leq \epsilon_{\text{state}} + \epsilon_{\text{ser}}$ in the $F_{\text{NIZK}^R_{\text{RECON}}}$ hybrid. Acceptance supplies a relation witness whose negative branch and carrier map must equal an authenticated singleton missing-token derivation. If p does not submit, no contribution from p exists. For the concrete verifier, add

AUTHENTICATED PRIVACY-PRESERVING DECK RECONSTRUCTION
the component package-extraction error ϵ_{KS} under A1, A4, and A5.

Proof. Suppose $\text{Veto}(p, m)$ occurs although m is not in p ’s authenticated set. Session authentication attributes the accepted submission to p . $F_{\text{NIZK}^R_{\text{RECON}}}$ acceptance supplies a witness containing the negative branch, residual carrier, and carrier-to-slot map. Exact-vector state binding requires the matching $(p, R_j, m, \text{epoch}, \text{missing-set})$ tuple in D_{prev} . If absent, either D_{prev} authentication failed or the implementation bytes refined to a different statement, giving $\epsilon_{\text{state}} + \epsilon_{\text{ser}}$. The concrete corollary additionally fails when an accepted component package has no extractable witness, contributing ϵ_{KS} . Overlapping carrier sets or pre-execution key leakage violate A8/A9 and fall outside the theorem.

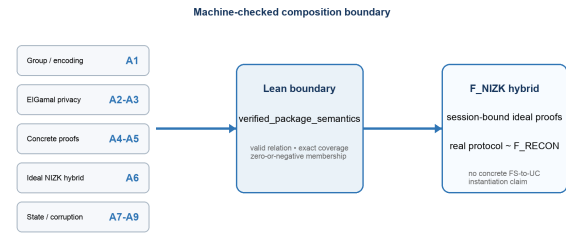


Figure 4. The formalization exposes a precise machine-checked boundary. Group algebra and component interfaces feed the composition theorem; computational assumptions are then used by the ideal-NIZK-hybrid argument.

7. Lean formalization

The Lean development separates the algebraic specification from the computational assumptions supplied by the implementation.

TABLE VII. Lean modules and checked results

Layer	File	Checked result
Residual lineage	ResidualCarrier Provenance.lean	Residual carrier from authenticated prior state
Reconstruction relation	Reconstruction.lean	Slot relation and unique removal semantics
Cross-key Sigma	Reconstruction JointSigma.lean	Cross-key relation, completeness, soundness, and HVZK
Slot OR	Reconstruction SlotOr.lean	Completeness, soundness, and algebraic HVZK
Composition boundary	Reconstruction Security.lean	End-to-end package semantics under component interfaces
Veto bound	Reconstruction Veto.lean	Non-owner veto impossibility and negligible error union bound

The algebraic Statement contains the aggregate key, the authenticated residual-key description, canonical cards, residual ciphertexts, and contributions. The Witness contains the removed-slot bitmap, contribution randomness, carrier-to-slot map, residual randomness, injectivity, and exact-coverage condition. Relation states the reveal-token subtraction equations and the per-slot contribution equations. The one-owner witness is the concrete specialization currently implemented by the AIR producer.

ComponentInterface records the external guarantees for the concrete hidden shuffle and linear proofs, the ideal NIZK hybrid interface, session/statement binding, Rust-to-Lean serialization, authenticated state, and cross-player disjointness. Reduction connects those guarantees to prove, verify, extraction, and view functions. The Lean theorem does not derive a Fiat–Shamir-to-UC realization.

The main composition theorem is `verified_package_semantics`. Given a `VerifiedPackage` containing a well-formed statement, an extracted witness, the algebraic relation, and a proof that the component interface holds, it derives in one theorem: (i) `ValidRelation` for the public statement and witness; (ii) exact correspondence between `removed i = true` and an authenticated residual-carrier index; and (iii) the zero-or-negative-card membership equation for every contribution.

The Rust–Lean refinement evidence is intentionally scoped but now covers complete statement and proof-instance wire schemas. Both sides first consume `paper/experiments/reconstruction_refinement_vector.json`, fixing version 3, epoch 11, an eight-card statement, two residual carriers, and bitmap 01001000. The

AUTHENTICATED PRIVACY-PRESERVING DECK RECONSTRUCTION `reconstruction_statement_schema_v3.json` fixture then freezes a 1,041-byte valid StarkCurve statement as eleven contiguous fields: version, context digest, epoch, prior-state digest, aggregate key, owner key, card count, cards, carrier count, residual carriers, and contributions. It also records a seeded proof instance as 3,829 bytes: version, negative-contribution count and vector, cross-key proofs, the complete Bayer–Groth multi-exponentiation and product substructure, slot-proof count, and all slot OR proofs. The proof section contains seven top-level fields and thirty nested field slices.

Rust regenerates the statement and exact seeded proof bytes with `ChaCha20`, verifies the proof, and checks every top-level and nested offset, length, and byte slice. Lean checks the semantic vector, the eleven-field statement schema, the seven-field proof schema, the 141-byte statement prefix, five-byte proof prefix, and both SHA-256 digests. A fresh production proof remains randomized, so this is a frozen proof-instance schema rather than a claim that all proof encodings are canonical. Truncation, trailing-field, digest-reordering, epoch-replay, foreign-card, and swapped-key mutations remain negative tests. Lean still does not claim that DDH, random-oracle security, or Bayer–Groth knowledge soundness follow from group algebra; those facts enter `ComponentInterface` and `Reduction` explicitly.

The repository’s Lean checks include a no-sorry audit and an axiom audit for the principal reconstruction results. The audit reports only the trusted Lean foundations used by imported libraries and no protocol-specific axiom.

8. Implementation and reproducibility

The code is organized as follows:

TABLE VIII. Implementation components

Component	Role
<code>poker-protocol-core</code>	Curve arithmetic, ElGamal, and transcripts.
<code>poker-protocol-bg</code>	Bayer–Groth shuffle component.
<code>poker-protocol-proofs</code>	Reconstruction, cross-key, OR, and related proofs.
<code>poker_protocol</code>	Native adapter, ABI, game integration, deadline policy, and settlement ledger boundary.
<code>client-wasm</code>	Browser bridge and reproducible WASM benchmark.
<code>poker_protocol_lean</code>	Formal specification and checked composition.

The native path uses the Stark-curve/Poseidon transcript domain. The Ristretto adapter constructs the public submission object; verification of an external AIR archive is outside this repository. The Move contract stores the partial ciphertext after subtracting submitted reveal tokens, while the AIR test helper derives the $|U| = 1$ owner-residual vector by subtracting every other seat's token. When two or more tokens are missing, no owner-residual vector entry is created and the rebuilt canonical slot remains unchanged. The paper and implementation repository is https://github.com/linqining/poker_protocol.

The repository also contains the pure Rust ``ReconstructionSubmissionPolicy`` and ``ReconstructionSettlementAdapter``. Policy tests cover full refund, deposit-capped penalties, timely compensation, duplicate/late/unknown/wrong-epoch rejection, idempotent finalization, the all-missed burn case, digest validation, and arithmetic overflow. The in-memory settlement ledger adds tests for exact escrow matching, idempotent lock and settlement retries, conflicting same-epoch proposals, malformed player arithmetic, noncanonical submitter order, inaccurate missed-submitter metadata, extra or missing deposits, and escrow/release overflow. The simulated chain host additionally tests deployment-digest binding, domain-separated signed-transaction admission, canonical sender/public-key binding, identity-key rejection, signature and proposal tamper rejection, transaction-hash binding for deployment, public key, nonce, gas, proposal, and signature, canonical PSTX-envelope roundtrip, bad magic/version, truncation, trailing bytes, unsigned encoding rejection, nonce admission and replay, insufficient gas, pending idempotence, confirmation-depth commit, and reorg rollback with nonce restoration. A production adapter must still bridge this reference envelope to its chain format and supply consensus, mempool, deployed verifier, and finality semantics.

To reproduce the checks:

```
./scripts/install_repro_deps.sh
./scripts/reproduce_paper.sh
```

The installer configures pinned Rust, Lean, Node.js, wasm-pack, Circom 2.2.3, and snarkjs 0.7.5 versions in user-writable locations without sudo; CIRCOM_BIN can select an explicit compatible compiler. The runner verifies the committed baseline and source hashes, executes the Rust, WASM, Lean, scoped-baseline, and refinement checks, and writes fresh timing grids under `.repro/results` rather than replacing the paper baselines. Each run records

the host, tool versions, Git state, and result hashes in `run_metadata.json`.

A reference native run uses StarkCurve, the production RECONSTRUCT_POSEIDON transcript domain, a release build, one untimed warm-up, and 30 timed samples per grid cell. For $n = 52$ and $k = 13$, proving has median/mean $\pm$ sample-sd/P95 157.0 / 157.9 $\pm$ 7.0 / 171.6 ms, while verification has 105.4 / 108.7 $\pm$ 9.7 / 126.6 ms. The proof is 21.78 KB; peak allocations are 85.2 KiB for proving and 39.4 KiB for verification. At $k = 26$, median proving and verification are 167.7 ms and 113.3 ms with a 24.69 KB proof. The full grid is committed at `paper/experiments/reconstruction_stark.csv`.

These timings describe only this reconstruction package on the recorded machine. They are not a head-to-head comparison with the trick-taking implementation of Bella et al. [48], whose published 52-card shuffle timings cover a different game relation, implementation stack, and hardware. No cross-implementation superiority claim is made.

TABLE IX. Native reconstruction benchmark: median / mean $\pm$ sample sd / P95, 30 release samples

n	k	Prove statistics	Verify statistics	Proof	Peak prove
13	1	36.4 / 36.4 $\pm$ 0.6 / 37.1 ms	25.0 / 25.2 $\pm$ 0.5 / 26.2 ms	5.37 KB	20.0 KiB
26	13	83.2 / 83.5 $\pm$ 2.3 / 90.3 ms	56.9 / 56.8 $\pm$ 0.9 / 58.6 ms	12.63 KB	46.5 KiB
52	13	157.0 / 157.9 $\pm$ 7.0 / 171.6 ms	105.4 / 108.7 $\pm$ 9.7 / 126.6 ms	21.78 KB	85.2 KiB
52	26	167.7 / 170.5 $\pm$ 7.4 / 191.1 ms	113.3 / 116.5 $\pm$ 10.4 / 139.8 ms	24.69 KB	93.0 KiB

Component profiling separates residual/setup and statement construction, cross-key proofs, Bayer–Groth shuffle, per-slot OR proofs, Borsh serialization, and the three corresponding verification stages. For $n = 52$ and $k = 13$, median proving components are 18.5, 12.0, 68.0, and 57.8 ms; verification components are 9.9, 48.0, and 43.9 ms. These are observational stage timings sampled in separate calls, so their sums may differ from the end-to-end timer. The complete grid is committed at `paper/experiments/reconstruction_components.csv`.

As a scoped fairness baseline, we compile a single-slot Circom relation with a private Boolean selector and public field values satisfying `out = selector*(-card)`, then run

Groth16 over BN128 with snarkjs 0.7.5. The measured circuit has 2 constraints, an 808-byte proof in the recorded run, 270.4 ms proving, and 228.6 ms verification. This baseline intentionally excludes ElGamal ciphertexts, residual decryption, hidden carrier-to-slot mapping, Bayer–Groth, cross-key proofs, authenticated state, and dropout composition; it is not a claim that Groth16 implements the complete protocol. The circuit, setup commands, unsupported-semantics list, and JSON result are in `paper/baselines` and `paper/experiments/circom_slot_baseline.json`.

A broader finite-domain Groth16 baseline models the central multi-slot semantics for the $n = 52$, $k = 13$ reference shape: each contribution is constrained to $\{0, -\text{card}_i\}$, each of the 13 private carriers maps to exactly one canonical slot, and each negative slot is covered exactly once. The branch bits and 13×52 one-hot carrier map are private. The measured circuit has 2197 constraints and 846 wires; witness generation, proving, and verification take 65.4, 282.9, and 222.3 ms, with an 807-byte proof and 1657-byte public-signal JSON. This is closer to reconstruction semantics than the single-slot baseline, but it still uses scalar card identifiers rather than curve ElGamal ciphertexts and excludes cross-key group equations, Bayer–Groth rerandomization, authenticated state, and protocol composition. The circuit and measured metadata are committed at `paper/baselines/circom_multislot_relation.circom` and `paper/experiments/circom_multislot_baseline.json`.

A stronger curve-level baseline instantiates the same curve-generic reconstruction relation on BN254 G1: ElGamal ciphertexts, cross-key negation proofs, Bayer–Groth rerandomized permutation, per-slot OR proofs, and exact residual-carrier coverage all remain present. The baseline uses SHA3 Fiat–Shamir rather than the production StarkCurve Poseidon domain. A new BN254 Borsh v3 codec serializes 32-byte compressed G1 points and canonical big-endian scalars; the measured proof/statement sizes are therefore 21,781/5,969 bytes rather than inferred. The stable precompile ABI accepts this BN254/BayerGrothSlotOr/SHA3 combination, and a native ABI verifier roundtrips encoded requests. At $n = 52$ and $k = 13$, 30 release samples after one untimed warm-up have BN254 median/P95 proving 158.2/167.9 ms and verification 101.2/110.7 ms; the production StarkCurve medians from the committed grid are 157.0 ms and 105.4 ms on the same host class. Roundtrip, truncation, tampering, wrong-domain, and plaintext-semantics tests cover the codec, ABI, and proof package. This is a

protocol-instantiation and native-ABI comparison, not an on-chain deployment or chain consensus claim.

The same Rust reconstruction implementation is compiled to wasm32 through client-wasm. The bridge returns a `BrowserReconstructionV3Bundle` containing the canonical Borsh statement and proof, decodes it, and verifies it with the production Poseidon transcript domain before reporting a row. In the 30-sample release Node/V8 run, $n = 52$ and $k = 13$ has proving median/mean $\pm$ sample-sd/P95 595.0 / 610.8 $\pm$ 40.2 / 728.0 ms, and verification 430.0 / 436.1 $\pm$ 20.9 / 478.0 ms; the proof is 21.78 KB and the statement-plus-proof bundle is 27.75 KB. Median WASM proving is 3.8 times the native result on this machine and remains below one second. The reference grid, commands, warm-up policy, environment, and hashes are recorded in `paper/experiments`. This grid remains a Node/V8 host measurement, not a browser-page measurement.

Two separate automated browser-page runs use Chrome 153.0.8010.53 on the recorded macOS host. The Chrome DevTools Protocol opens an isolated temporary profile, serves the web-target package from a local server, and waits for the same ten-cell result in HeadlessChrome and headed modes. For $n = 52$ and $k = 13$, HeadlessChrome has 30-sample median/P95 proving 429.0/437.0 ms and verification 311.0/316.0 ms; headed Chrome has 425.0/434.0 and 314.0/319.0 ms. A manually opened user-normal Safari 18.3 page on the same recorded host completes the identical grid; its median/P95 proving and verification at $n = 52$, $k = 13$ are 294.0/298.0 and 216.0/219.0 ms. Module initialization after page script start is 15.5 ms in headless Chrome, 26.2 ms in headed Chrome, and 25.0 ms in desktop Safari. The Chrome runs are automated isolated-profile measurements; Safari is a user-normal desktop measurement. None of these runs is Android, iOS, full-network-cold-load, cache-state, or peak-memory coverage.

A separate HeadlessChrome run repeats the ten-cell, 30-sample grid with Chrome DevTools Protocol `Performance.getMetrics` polling every 100 ms and a 20 ms yield between cells. It observes 1,432 renderer samples, with peak used/total JS heap 35.8/56.2 MiB. Sampling and yields perturb scheduling, so this is not a fourth timing run. It measures only the renderer JavaScript heap, not browser-process RSS, native/WASM-process peak allocation, Safari, Android, or iOS memory.

A separate cache-disabled loopback experiment performs 30 independent page loads. Before each load, CDP clears

and disables the browser cache; the page then downloads HTML, JavaScript, and the 243,642-byte WASM module from a local HTTP/1.1 server and executes one $n = 52$, $k = 13$ proof. Browser-reported transfer is exactly 264,119 bytes per run, while the server writes 263,219 body bytes; the 900-byte difference is accounted for by response headers. Median/P95 WASM resource duration is 1.8/3.3 ms, module initialization is 13.2/23.5 ms, and the one-sample proof/verify pair is 411/302 ms median. This controls code-download and cache state, but it is loopback only: proof upload, TCP cold start, LAN, Internet, and peak memory remain outside its claims.

The upload variant extends this controlled loopback experiment to the actual proof package. A new WASM export generates one $n = 52$, $k = 13$ statement/proof bundle, verifies and Borsh-decodes it, and returns the canonical 27,750-byte wire bundle. The page sends those bytes as an HTTP/1.1 binary POST; the Node server decodes the bundle and runs the production WASM verifier before returning its SHA-256. Across 30 cache-disabled runs, median/P95 upload wall time is 473.3/520.5 ms. Browser download transfer is 273,389 bytes, the server receives that code plus the 27,750-byte proof bundle, and every upload is accepted only after verification. Resource Timing reports a 432-byte transfer for the upload response separately from the 27,750-byte request body. This closes the local wire-upload boundary, but still does not model cold TCP setup, TLS, LAN transport, Internet transport, or service workers.

A separate service-worker experiment installs a same-origin cache for only the benchmark HTML, JavaScript, and 247,743-byte WASM module before the measured loads; proof upload and result submission always bypass that cache. Across 30 controlled warm loads, every run reports zero Resource Timing transfer for HTML, JavaScript, and WASM, while the server observes exactly the 27,750-byte proof upload and verifies it. Median/P95 WASM resource duration is 1.1/3.9 ms, module initialization is 15.4/30.0 ms, and upload wall time is 499.0/606.5 ms. This is a warm service-worker loopback measurement, not a cold-install measurement, browser HTTP-cache study, TLS/LAN/Internet path, or offline-availability claim.

A cold-install companion uses a unique page scope, service-worker script query, and Cache API name for each of 30 runs. Browser HTTP cache is cleared and disabled, and the server observes exactly five requests per run: the cold page, worker script, upload HTML, JavaScript, and

247,743-byte WASM body. It writes 273,188 bytes into Cache Storage and receives 279,716 body bytes in total. Median/P95 registration is 30.3/45.7 ms, active-ready is 35.9/61.5 ms, and total measured installation is 41.2/64.2 ms. This measures first installation in isolated scopes, not update invalidation, browser HTTP-cache reuse, TLS, LAN, or Internet transport.

An update-invalidations companion keeps the worker URL, scope, and cache name fixed within each run while changing the worker bytes from version 1 to version 2. The server records both worker versions, both generations fetch the three static assets, and the page observes controller version 1 then 2 before reporting an activated controller. Across 30 unique-scope updates, initial registration median/P95 is 49.3/71.0 ms; ``registration.update()`` through activation and cache inspection is 1015.9/1091.3 ms; and total measured update is 1066.2/1149.0 ms. Each generation writes 273,188 cached bytes and the server sends 557,084 bytes across nine observations. This measures same-URL byte-update invalidation with browser HTTP cache disabled, not normal browser HTTP-cache reuse, TLS, LAN, or Internet transport.

A separate no-service-worker companion measures normal browser HTTP-cache reuse. The server marks the benchmark HTML, JavaScript, and 247,743-byte WASM module ``public,max-age=31536000,immutable``; one seed load clears and populates the cache, and each of 30 measured loads then uses the same stable URLs with browser HTTP cache enabled. Every measured run reports zero Resource Timing transfer for all three assets, while the server observes only the 27,750-byte proof upload and verifies it. Median/P95 WASM resource duration is 0.7/1.3 ms, module initialization is 9.7/11.7 ms, and upload wall time is 512.3/575.0 ms. This measures ordinary HTTP-cache reuse, not service-worker Cache Storage, TLS, LAN, or Internet transport.

A manual user-normal Safari 18.3 companion repeats the ordinary browser-cache experiment on the same immutable loopback origin. Across 30 measured loads after one seed, every HTML/JavaScript/WASM transfer is zero, each proof upload is the same 27,750-byte package, and the server verifies it. Median/P95 WASM resource duration is 1.0/2.0 ms, module initialization is 3.0/4.0 ms, and upload wall time is 505.0/520.0 ms. This provides desktop cross-browser HTTP-cache evidence, not mobile-device, TLS, LAN, Internet, or service-worker behavior.

TABLE XI. Browser-page benchmarks on macOS, median / P95, 30 timed samples

n	k	Mode	Prove ms	Verify ms	Bundle
13	1	Headless Chrome	103.0 / 105.0	76.0 / 77.0	6.82 KB
13	1	Headed Chrome	103.0 / 113.0	76.0 / 77.0	6.82 KB
13	1	Desktop Safari	73.0 / 75.0	54.0 / 56.0	6.82 KB
26	13	Headless Chrome	238.0 / 242.0	174.0 / 177.0	16.10 KB
26	13	Headed Chrome	237.0 / 241.0	174.0 / 176.0	16.10 KB
26	13	Desktop Safari	167.0 / 171.0	122.0 / 138.0	16.10 KB
52	1	Headless Chrome	388.0 / 394.0	285.0 / 288.0	24.29 KB
52	1	Headed Chrome	382.0 / 391.0	283.0 / 287.0	24.29 KB
52	1	Desktop Safari	267.0 / 276.0	198.0 / 201.0	24.29 KB
52	13	Headless Chrome	429.0 / 437.0	311.0 / 316.0	27.75 KB
52	13	Headed Chrome	425.0 / 434.0	314.0 / 319.0	27.75 KB
52	13	Desktop Safari	294.0 / 298.0	216.0 / 219.0	27.75 KB
52	26	Headless Chrome	470.0 / 478.0	347.0 / 351.0	31.49 KB
52	26	Headed Chrome	467.0 / 474.0	342.0 / 346.0	31.49 KB
52	26	Desktop Safari	327.0 / 343.0	239.0 / 245.0	31.49 KB

To separate transport from page-version effects, the current upload page was also rerun once over HTTP and once over TLS. Both 30-run pairs clear the browser cache, download the same 274,088 browser-transfer bytes, receive the same 300,938 server-side bytes, and upload the same verified 27,750-byte bundle. Median/P95 upload wall time is 544.3/706.9 ms over HTTP and 549.5/771.0 ms over TLS on this loopback host. The TLS run uses a temporary self-signed RSA-2048 127.0.0.1 certificate and Chrome certificate-error bypass, so it measures local TLS transport rather than public PKI, LAN, or Internet behavior.

TABLE X. WASM reconstruction benchmark on Node/V8 with 30 timed samples

n	k	Prove: median / mean $\pm$ sd / P95	Verify: median / mean $\pm$ sd / P95	Proof	Total
13	1	147.0 / 152.1 $\pm$ 13.2 / 186.0 ms	106.0 / 107.1 $\pm$ 6.2 / 109.0 ms	5.37 KB	6.82 KB
26	13	350.0 / 353.2 $\pm$ 17.1 / 398.0 ms	245.0 / 246.2 $\pm$ 10.0 / 273.0 ms	12.63 KB	16.10 KB
52	13	595.0 / 610.8 $\pm$ 40.2 / 728.0 ms	430.0 / 436.1 $\pm$ 20.9 / 478.0 ms	21.78 KB	27.75 KB
52	26	662.0 / 684.9 $\pm$ 55.3 / 830.0 ms	467.0 / 473.9 $\pm$ 25.4 / 517.0 ms	24.69 KB	31.49 KB

TABLE XII. Measured native component attribution for n=52 and k=13

Stage	Prove cost	Verify cost	Interpretation
Residual and statement setup	18.5 ms	—	State-derived carriers and public statement construction.
Cross-key proofs	12.0 ms	9.9 ms	Cost attributable to k carrier-link relations.
Bayer-Groth	68.0 ms	48.0 ms	Cost attributable to hidden permutation and rerandomization.
Slot OR proofs	57.8 ms	43.9 ms	Cost attributable to n slot-membership relations.
Serialization	1.0 ms	—	Borsh statement and proof encoding only.

The component table is a measured stage attribution, not a claim that the protocol remains secure when a component is removed. It answers the ablation cost question by showing the wall-time contribution of cross-key, shuffle, slot-OR, and serialization stages. The closest Mental Poker comparison remains an operation-count comparison because its authors do not publish compatible runtime or proof-byte measurements.

9. Limitations and future work

- A malicious player that never submits is an availability event. Deadlines, deposits, or a replacement submitter are needed for an application-level policy.
- Authenticated reveal-token lineage is essential. Without the residual derivation, the veto theorem does not hold.
- The deck size, owner-residual carrier count, public keys, canonical cards, epoch, and state digest are public; the protocol does not hide these metadata.
- A carrier-specific $|U| \geq 2$ residual takes the normal zero branch and the corresponding canonical card remains in the new deck. Threshold removal of such a jointly unknown card is a future protocol extension and is not required for the present reconstruction guarantee.
- The UC theorem is an $F_NIZK^R_RECON$ -hybrid result. The current cumulative Poseidon Fiat–Shamir transcript is not claimed to realize concurrent UC NIZK.
- The Node/V8, automated Chrome, and one desktop Safari timing results do not cover Android, iOS, other Safari versions, public PKI, LAN, Internet latency, or remote cold-connection behavior. The TLS experiment uses a temporary self-signed certificate and certificate-error bypass over loopback. Service-worker and HTTP-cache evidence covers isolated cold installation, same-URL byte-update invalidation, warm Cache Storage hits, immutable Chrome HTTP-cache reuse, and manual desktop Safari HTTP-cache reuse over loopback; it does not cover cross-browser SW update behavior or remote transport. Chrome memory coverage is sampled renderer-JS-heap only and does not provide browser-process RSS or mobile peak memory.

- Adaptive corruption requires erasures or non-committing techniques.
- Multiple authorization of the same owner-residual carrier is excluded by the authenticated missing-key invariant.

10. Conclusion

The reconstruction protocol addresses a liveness gap in cooperative Mental Poker. Authenticated reveal-token subtraction yields a residual ciphertext carrier for every card: owner-residual when one token is missing, jointly keyed when several tokens are missing. Owner-residual carriers authorize exact removals; a jointly keyed carrier authorizes no negative branch, so the corresponding canonical card remains in the freshly encrypted deck. The active table can therefore continue even when no individual player knows an old card plaintext. The proof system combines hidden mapping, carrier-to-plaintext binding, per-slot zero-or-negative semantics, and state/transcript binding in one auditable package.

An accepted ideal-NIZK submission has a witness with exact residual-carrier coverage; the concrete implementation has the same semantic consequence only under the stated component extraction assumptions. Authenticated lineage and missing-key binding then prevent a player from vetoing another player’s card except with the stated state, refinement, and concrete-extraction errors. The Lean layer connects the algebraic relation to this explicit boundary without claiming a verified concurrent Fiat–Shamir realization.

Declarations

Code and reproducibility. The implementation, Lean development, benchmark grids, measurement metadata, baseline circuit, and reproduction scripts are available at https://github.com/linqining/poker_protocol. The committed native and WASM measurements are in `reconstruction_stark.csv` and `reconstruction_wasm.csv`; their environment, commands, hashes, and limits are recorded in `benchmark_metadata.json`. The reproduction runner writes fresh measurements separately rather than overwriting the committed baselines.

Data availability. No proprietary or third-party experimental data were used. The paper’s measurement files and canonical semantic fixture are included in the repository.

License. This article is distributed under arXiv.org perpetual, non-exclusive license 1.0. The repository implementation is licensed separately under BUSL-1.1.

Conflict of interest. The authors declare no conflict of interest.

Funding. No funding was declared for this work.

Author contributions. Conceptualization, Methodology, Writing – original draft

AI assistance. AI-assisted text and workflow tools (Codex/GPT-5) were used for language editing, manuscript preparation, and reproducibility support. The author reviewed and takes full responsibility for all technical claims, proofs, data, code, and final content.

Appendix A. Relation to dropout-tolerant mental poker

Castellà-Roca, Sebé, and Domingo-Ferrer study dropout-tolerant Mental Poker without a trusted third party and use zero-knowledge techniques to let a game continue after a player leaves [7]. Their construction is a relevant reference for liveness, but its prover-side authorization does not bind each removal to authenticated per-card owner-residual lineage. A prover may therefore be able to veto a card that is not its own. The present protocol treats exclusion of that behavior as a separate security goal: every accepted negative branch must correspond exactly to an authenticated singleton missing-token derivation for the submitting owner.

The protocol abstractions and security boundaries therefore differ. Our construction derives a residual carrier from an authenticated, state-bound reveal-token transcript, proves exact coverage for the owner-residual subset, hides the carrier-to-slot mapping, and enforces a per-slot zero-or-negative plaintext relation. Only one missing token yields an owner-residual carrier that one participant can decrypt and use to authorize removal. Several missing tokens yield a jointly keyed carrier whose canonical card is retained without revealing its plaintext. Accordingly, [7] supports the dropout-tolerance motivation but does not establish the residual-carrier semantics, non-owner-veto bound, or ideal-NIZK-hybrid composition theorem stated here.

References

- [1] R. Canetti. “Universally Composable Security: A New Paradigm for Cryptographic Protocols.” In IEEE FOCS, pp. 136–145, 2001. doi:10.1109/SFCS.2001.959888.
- [2] S. Bayer and J. Groth. “Efficient Zero-Knowledge Argument for Correctness of a Shuffle.” In EUROCRYPT, LNCS 7237, pp. 263–280, 2012. doi:10.1007/978-3-642-29011-4_17.
- [3] D. Chaum and T. P. Pedersen. “Wallet Databases with Observers.” In CRYPTO, LNCS 740, pp. 89–105, 1992. doi:10.1007/3-540-48071-4_7.
- [4] R. Cramer, I. Damgard, and B. Schoenmakers. “Proofs of Partial Knowledge and Simplified Design of Witness Hiding Protocols.” In CRYPTO, LNCS 839, pp. 174–187, 1994. doi:10.1007/3-540-48658-5_19.
- [5] A. Fiat and A. Shamir. “How To Prove Yourself: Practical Solutions to Identification and Signature Problems.” In CRYPTO, LNCS 263, pp. 186–194, 1986. doi:10.1007/3-540-47721-7_12.
- [6] C.-P. Schnorr. “Efficient Identification and Signatures for Smart Cards.” In CRYPTO, LNCS 435, pp. 239–252, 1989. doi:10.1007/0-387-34805-0_22.
- [7] J. Castella-Roca, F. Sebe, and J. Domingo-Ferrer. “Dropout-Tolerant TTP-Free Mental Poker.” In Trust, Privacy, and Security in Digital Business, LNCS 3592, pp. 30–40, 2005. doi:10.1007/11537878_4.
- [8] J. Castella-Roca. “Contributions to Mental Poker.” PhD thesis, Universitat Autònoma de Barcelona, 2005.
- [9] A. Barnett and N. P. Smart. “Mental Poker Revisited.” In Cryptography and Coding, LNCS 2898, pp. 370–383, 2003. doi:10.1007/978-3-540-40974-8_29.
- [10] K. Kurosawa, Y. Katayama, and W. Ogata. “Reshuffable and Laziness Tolerant Mental Card Game Protocol.” IEICE Transactions on Fundamentals, 1997.
- [11] W. H. Soo, A. Samsudin, and A. Goh. “Efficient Mental Card Shuffling via Optimised Arbitrary-Sized Benes Permutation Network.” In Information Security, LNCS 2433, pp. 446–458, 2002. doi:10.1007/3-540-45811-5_35.
- [12] I. Bentov, R. Kumaresan, and A. Miller. “Instantaneous Decentralized Poker.” In ASIACRYPT, LNCS 10625, pp. 410–440, 2017. doi:10.1007/978-3-319-70697-9_15.
- [13] B. David, R. Dowsley, and M. Larangeira. “Kaleidoscope: An Efficient Poker Protocol with Payment Distribution and Penalty Enforcement.” In Financial Cryptography and Data Security, LNCS 10958, pp. 500–519, 2018. doi:10.1007/978-3-662-58387-6_27.
- [14] B. David, R. Dowsley, and M. Larangeira. “ROYALE: A Framework for Universally Composable Card Games with Financial Rewards and Penalties Enforcement.” In Financial Cryptography and Data Security, LNCS 11598, pp. 282–300, 2019. doi:10.1007/978-3-030-32101-7_18.
- [15] T. ElGamal. “A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms.” IEEE Transactions on Information Theory, vol. 31, no. 4, pp. 469–472, 1985. doi:10.1109/TIT.1985.1057074.
- [16] S. Goldwasser, S. Micali, and C. Rackoff. “The Knowledge Complexity of Interactive Proof Systems.” SIAM Journal on Computing, vol. 18, no. 1, pp. 186–208, 1989. doi:10.1137/0218012.
- [17] M. Blum, P. Feldman, and S. Micali. “Non-Interactive Zero-Knowledge and Its Applications.” In ACM STOC, pp. 103–112, 1988. doi:10.1145/62212.62222.
- [18] M. Bellare and P. Rogaway. “Random Oracles Are Practical: A Paradigm for Designing Efficient Protocols.” In ACM CCS, pp. 62–73, 1993. doi:10.1145/168588.168596.
- [19] D. Pointcheval and J. Stern. “Security Proofs for Signature Schemes.” In EUROCRYPT, LNCS 1070, pp. 387–398, 1996. doi:10.1007/3-540-68339-9_33.
- [20] M. Fischlin. “Communication-Efficient Non-Interactive Proofs of Knowledge with Online Extractors.” In CRYPTO, LNCS 3621, pp. 152–168, 2005. doi:10.1007/11535218_10.

- [21] D. Unruh. “The Fiat-Shamir Transformation in a Quantum World.” In ASIACRYPT, LNCS 8270, pp. 1–18, 2013. doi:10.1007/978-3-642-42045-0_4.
- [22] J. Furukawa and K. Sako. “An Efficient Scheme for Proving a Shuffle.” In CRYPTO, LNCS 2139, pp. 368–387, 2001. doi:10.1007/3-540-44647-8_22.
- [23] C. A. Neff. “A Verifiable Secret Shuffle and Its Application to E-Voting.” In ACM CCS, pp. 116–125, 2001. doi:10.1145/501983.502000.
- [24] J. Groth. “A Verifiable Secret Shuffle of Homomorphic Encryptions.” *Journal of Cryptology*, vol. 23, pp. 546–579, 2010. doi:10.1007/s00145-010-9067-9.
- [25] D. Wikström. “A Universally Composable Mix-Net.” In TCC, LNCS 2951, pp. 317–335, 2004. doi:10.1007/978-3-540-24638-1_18.
- [26] J. Groth and A. Sahai. “Efficient Non-Interactive Proof Systems for Bilinear Groups.” In EUROCRYPT, LNCS 4965, pp. 415–432, 2008. doi:10.1007/978-3-540-78967-3_19.
- [27] M. Belenkiy, M. Chase, C. C. Erway, J. Jannotti, A. Kupcu, A. Lysyanskaya, and E. Rachlin. “Simulation-Sound NIZK Proofs for a Practical Language and Constant Size Group Signatures.” In ASIACRYPT, LNCS 4284, pp. 444–459, 2006. doi:10.1007/11935230_29.
- [28] G. Barthe, B. Gregoire, S. Heraud, and S. Zanella Beguelin. “Computer-Aided Cryptographic Proofs.” In ITP, LNCS 7406, pp. 11–27, 2012. doi:10.1007/978-3-642-33125-1_1.
- [29] A. Lochbihler. “CryptHOL: Game-Based Proofs in Higher-Order Logic.” *Journal of Cryptology*, vol. 33, pp. 494–566, 2020. doi:10.1007/s00145-019-09341-z.
- [30] J. B. Almeida, M. Barbosa, G. Barthe, A. Blot, B. Gregoire, V. Laporte, T. Oliveira, H. Pacheco, B. Schmidt, and P.-Y. Strub. “Jasmin: High-Assurance and High-Speed Cryptography.” In ACM CCS, pp. 1807–1823, 2017. doi:10.1145/3133956.3134078.
- [31] J.-K. Zinzindohoue, K. Bhargavan, J. Protzenko, and B. Beurdouche. “HACL*: A Verified Modern Cryptographic Library.” In ACM CCS, pp. 1789–1806, 2017. doi:10.1145/3133956.3134043.
- [32] K. Bonawitz et al. “Practical Secure Aggregation for Privacy-Preserving Machine Learning.” In ACM CCS, pp. 1175–1191, 2017. doi:10.1145/3133956.3133982.
- [33] J. H. Bell, K. A. Bonawitz, A. Gascon, T. Lepoint, and M. Raykova. “Secure Single-Server Aggregation with (Poly)Logarithmic Overhead.” In ACM CCS, pp. 1253–1269, 2020. doi:10.1145/3372297.3417885.
- [34] D. Mouris and N. G. Tsoutsos. “Zilch: A Framework for Deploying Transparent Zero-Knowledge Proofs.” *IEEE TIFS*, vol. 16, pp. 3269–3284, 2021. doi:10.1109/TIFS.2021.3074869.
- [35] A. Bay, Z. Erkin, J.-H. Hoepman, S. Samardjiska, and J. Vos. “Practical Multi-Party Private Set Intersection Protocols.” *IEEE TIFS*, vol. 17, pp. 1–15, 2022. doi:10.1109/TIFS.2021.3118879.
- [36] B. Mennink. “Secure Distributed Modular Exponentiation: Systematic Analysis and New Results.” *IEEE TIFS*, vol. 18, pp. 4188–4197, 2023. doi:10.1109/TIFS.2023.3293396.
- [37] X. Cao, Z. Yang, J. Ning, C. Jin, R. Lu, Z. Liu, and J. Zhou. “Dynamic Group Time-Based One-Time Passwords.” *IEEE TIFS*, vol. 19, pp. 4897–4913, 2024. doi:10.1109/TIFS.2024.3386350.
- [38] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastien. “Bringing the Web up to Speed with WebAssembly.” In ACM PLDI, pp. 185–200, 2017. doi:10.1145/3062341.3062363.
- [39] J. Groth. “On the Size of Pairing-Based Non-Interactive Arguments.” In EUROCRYPT, LNCS 9666, pp. 305–326, 2016. doi:10.1007/978-3-662-49896-5_11.
- [40] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza. “Zerocash: Decentralized Anonymous Payments from Bitcoin.” In *IEEE Symposium on Security and Privacy*, pp. 459–474, 2014. doi:10.1109/SP.2014.36.
- [41] A. Shamir, R. L. Rivest, and L. M. Adleman. “Mental Poker.” In *The Mathematical Gardner*, pp. 37–43. Prindle, Weber, and Schmidt, 1981. doi:10.1007/978-1-4684-6686-7_5.
- [42] S. Goldwasser and S. Micali. “Probabilistic Encryption and How to Play Mental Poker Keeping Secret All Partial Information.” In *ACM STOC*, pp. 365–377, 1982. doi:10.1145/800070.802212.
- [43] B. David, R. Dowsley, and M. Larangeira. “21 - Bringing Down the Complexity: Fast Composable Protocols for Card Games Without Secret State.” In *ACISP*, LNCS 11046, pp. 45–63, 2018. doi:10.1007/978-3-319-93638-3_4.
- [44] T. Oozu, M. Ishii, and K. Tanaka. “Dropout-Tolerant Mental Poker Protocol with Small Deposit and Optimal Upper Bound on Number of Dropouts.” In *Symposium on Cryptography and Information Security*, 2020.
- [45] T.-J. Wei. “Secure and Practical Constant Round Mental Poker.” *Information Sciences*, vol. 280, pp. 386–396, 2014. doi:10.1016/j.ins.2014.02.151.
- [46] T.-J. Wei. “Communication Efficient Shuffle for Mental Poker Protocols.” *Information Sciences*, vol. 181, no. 22, pp. 4661–4671, 2011. doi:10.1016/j.ins.2011.06.018.
- [47] X. Bultel and P. Lafourcade. “Secure Trick-Taking Game Protocols: How to Play Online Spades with Cheaters.” In *Financial Cryptography and Data Security*, 2019. <https://eprint.iacr.org/2019/375>.
- [48] R. Bella, X. Bultel, C. Chevalier, P. Lafourcade, and C. Olivier-Anclin. “Practical Construction for Secure Trick-Taking Games Even with Cards Set Aside.” In *Financial Cryptography and Data Security*, LNCS 13955, pp. 166–181, 2023. doi:10.1007/978-3-031-47754-6_10.
- [49] D. Beaver. “Extending Mental Poker.” *Cryptology ePrint Archive*, Paper 2025/1821, 2025. <https://eprint.iacr.org/2025/1821>.